

Task 1: CRUD tasks on listingsAndReviews Document Collection

Statement 1

1A: Display 3-bedroom apartments in Barcelona.

Compass

```
{ "address.market": "Barcelona", "bedrooms": 3, "property_type": "Apartment" }
```

Mongo Shell

```
db.listingsAndReviews.find({ "address.market": "Barcelona", "bedrooms": 3, "property_type": "Apartment" }).count()
```

```
db.listingsAndReviews.find({ "address.market": "Barcelona", "bedrooms": 3, "property_type": "Apartment" })
```

1B: listings in Barcelona that has both Wifi and Cable TV.

Compass

Query: { "address.market": "Barcelona", "amenities": { "\$all": ["Wifi", "Cable TV"] } }

Project: {amenities:1}

Mongo Shell

//To count documents

```
db.listingsAndReviews.find({ "address.market": "Barcelona", "amenities": { $all: ["Wifi", "Cable TV"] } }).count()
```

//To display AMENITIES

```
db.listingsAndReviews.find({ "address.market": "Barcelona", "amenities": { $all: ["Wifi", "Cable TV"] } }, { "_id": 1, "amenities": 1 }).pretty()
```

1C: Compute and display the average daily rate for Barcelona properties.

Compass

Go to the aggregations tab.

Stage 1: Make “match” stage.

```
{ "address.market": "Barcelona" }
```

Stage 2: Make “group” stage.

```
{ _id: null, average_daily_rate: { $avg: "$price" } }
```

Stage 3: \$project

```
{ _id: 0, average_daily_rate: 1 }
```

Mongo Shell

```
db.listingsAndReviews.aggregate([
  { $match: { "address.market": "Barcelona" } },
  { $group: { _id: null, // Group all documents together
    avgPrice: { $avg: "$price" } } // Calculate the average price
  }
])
```

1D: Display the average price for 3-bedroom apartments in Barcelona.

Compass

Go to the aggregations tab. Make 2 stages.

Stage 1: Make a stage “Match”

```
{ "address.market": "Barcelona", bedrooms: 3, property_type: "Apartment" }
```

Stage 2: Make a stage “Group”

```
{ _id: null, average_price: { $avg: "$price" } }
```

Stage 3: \$project

```
{ _id: 0, average_price: 1 }
```

Shell

```
db.listingsAndReviews.aggregate([
  { $match: { "address.market": "Barcelona", "bedrooms": 3, "property_type": "Apartment" } },
  { $group: { _id: null, average_price: { $avg: "$price" } } },
  { $project: { _id: 0, average_price: "$average_price" } }
]);
```

1E: listings within a \$10-range from the average price you found in 1D

Mongo Shell

```
db.listingsAndReviews.aggregate([
  // Stage 1: Match
  {
    $match: { "address.market": "Barcelona",
      "bedrooms": 3, "price": { $type: "decimal" }
    }
  },
  // Stage 2: Group
  { $group: { _id: null, avgPrice: { $avg: "$price" } } },
  // Stage 3: Add Fields
  {
    $addFields: {
      minPrice: { $subtract: [ "$avgPrice", 5 ] },
      maxPrice: { $add: [ "$avgPrice", 5 ] }
    }
  },
  // Stage 4: Lookup
  {
    $lookup: {
      from: "listingsAndReviews",
      let: {
        minPrice: "$minPrice",
        maxPrice: "$maxPrice"
      },
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $gt: [ "$price", "$$minPrice" ] },
                { $lt: [ "$price", "$$maxPrice" ] }
              ]
            }
          }
        }
      ]
    }
  }
]);
```

```

    ],
    as: "listingsWithinRange"
  }
},
// Stage 5: Project
{ $project: {averagePrice: "$avgPrice", listingsWithinRange: 1} }
]);

```

To count number of listings

```

db.listingsAndReviews.aggregate([
  // Stage 1: Match
  {
    $match: {
      "address.market": "Barcelona",
      "bedrooms": 3,
      "price": { $type: "decimal" }
    }
  },
  // Stage 2: Group
  {
    $group: {
      _id: null,
      avgPrice: { $avg: "$price" }
    }
  },
  // Stage 3: Add Fields
  {
    $addFields: {
      minPrice: { $subtract: ["$avgPrice", 5] },
      maxPrice: { $add: ["$avgPrice", 5] }
    }
  },
  // Stage 4: Lookup
  {
    $lookup: {
      from: "listingsAndReviews",
      let: {minPrice: "$minPrice", maxPrice: "$maxPrice"},
      pipeline: [
        {
          $match: {
            $expr: {
              $and: [
                { $gt: ["$price", "$$minPrice"] },
                { $lt: ["$price", "$$maxPrice"] }
              ]
            }
          }
        }
      ]
    }
  },
  as: "listingsWithinRange"
}

```

```

    },
    // Stage 5: Project
    {
      $project: { averagePrice: "$avgPrice",
        listingsWithinRangeCount: { $size: "$listingsWithinRange" }, listingsWithinRange: 1
      }
    }
  ]);

```

Note:

listingsWithinRangeCount: // Number of objects in listingsWithinRange

listingsWithinRange: 1 // Keep the array if you still want to display it

Mongo Compass

Go to the aggregations tab. Make 5 stages.

Stage 1: \$match

```
{ "address.market": "Barcelona", bedrooms: 3, price: { $type: "decimal" } }
```

Stage 2: \$Group

```
{ _id: null, avgPrice: { $avg: "$price" } }
```

Stage 3: \$addFields

```
{
  minPrice: { $subtract: [ "$avgPrice", 5 ] },
  maxPrice: { $add: [ "$avgPrice", 5 ] }
}
```

Stage 4: \$lookup

```
{
  from: "listingsAndReviews",
  let: { minPrice: "$minPrice", maxPrice: "$maxPrice" },
  pipeline: [
    {
      $match: {
        $expr: {
          $and: [
            { $gt: [ "$price", "$$minPrice" ] },
            { $lt: [ "$price", "$$maxPrice" ] }
          ]
        }
      }
    }
  ],
  as: "listingsWithinRange"
}
```

Stage 5: \$project

```
{ averagePrice: "$avgPrice", listingsWithinRange: 1 }
```

Explanation

Stage 1: \$match

This stage filters the documents to include only 3-bedroom apartments in Barcelona and specifies that the price should be of type decimal to ensure accurate calculations for the average price, preventing errors with non-decimal values.

- **Specifying the price type as decimal ensures that the average price calculation is precise** and avoids any inaccuracies that might occur if non-decimal types were present, particularly when dealing with currency.

Stage 2: \$group

This stage groups the filtered results into a single document and calculates the average price of the listings.

Stage 3: \$addFields

This stage adds new fields to the output document, specifically minPrice and maxPrice, which are calculated by subtracting and adding \$5 to the average price, respectively.

- **let:** Defines variables (like minPrice and maxPrice) that can be used in the subsequent pipeline stages for filtering.
- **pipeline:** This allows me in the \$lookup stage to do multiple filtering, projecting and grouping operations. Essentially allowing me to do complex operations by chaining matching and project together like in the aggregation pipeline.
- **\$expr:** Allows the use of aggregation expression in the \$match stage (\$gt and \$lt), enabling the comparison of the listing prices against the defined variables (i.e., minPrice and maxPrice).

Stage 4: \$lookup

This stage performs a join with the listingsAndReviews collection using the defined bounds for filtering listings that fall within the calculated price range.

Stage 5: \$project

This stage reshapes the output by including the average price and the listings within the specified price range, making it clear and concise.

1F: Display 10 “least-expensive” tourist destinations for travelers.

Shell

```
db.listingsAndReviews.aggregate([
  // Stage 1: Group
  { $group: { _id: "$address.market", average_price: { $avg: "$price" } } },
  // Stage 2: Sort
  { $sort: { average_price: 1 } },
  // Stage 3: Limit
  { $limit: 10 },
  // Stage 4: Project
  {
    $project: {
      market: "$_id",
      average_price: 1,
      _id: 0
    }
  }
]);
```

Compass

Go to the aggregations tab. Make 3 stages.

Stage 1: Make a \$group stage.

```
{
  _id: "$address.market",
  averagePrice: { $avg: "$price" }
}
```

Stage 2: Make a \$sort stage.

```
{averagePrice: 1}
```

Stage 3: Make a \$limit stage.

10

Stage 4 project

```
{ market: "$_id", averagePrice: 1, _id: 0 }
```

Note:

- Groups the documents by the market sub-field of the address and calculates the average price for each destination.
- Stage 2 sorts in ascending order where smaller values come first. -1 would be descending order. I want to have least expensive. If I had -1 then 10 most expensive would be at shown.

Statement 2 (Checked)

Compass

Step 1: In the document tab filter.

```
{ "name": "Be Happy in Porto" }
```

Step 2: After searching, click "Update". Then enter the following:

```
{ $addToSet: { amenities: "Netflix" } }
```

NOTE: The \$addToSet operator prevents duplicates. This ensures that "Netflix" is only added to the amenities array if it is not already present

TO TEST if shell works

Step 1: In the document tab filter.

```
{ "name": "Be Happy in Porto" }
```

Step 2: After searching, click "Update". Then enter the following:

```
{ $pull: { amenities: "Netflix" } }
```

Shell

```
db.listingsAndReviews.updateOne(  
  { name: "Be Happy in Porto" },  
  { $addToSet: { amenities: "Netflix" } }  
)
```

Statement 3 (Checked)

Shell

```
db.listingsAndReviews.updateOne(  
  { "name": "Be Happy in Porto" },  
  {  
    "$push": {  
      "reviews": {  
        "_id": "58433009",  
        "listing_id": "10083468",  
        "reviewer_id": "reviewer_id_placeholder",  
        "reviewer_name": "Aqua",  
        "date": new Date(),
```

```
"comments": "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there."
```

```
    }  
  }  
}  
);
```

To search in SHELL the comment

```
{ "_id": "58433009" }
```

Mongo Compass

First use the filter enter:

```
{ "name": "Be Happy in Porto" }
```

Then click "Update". Enter the following:

```
{  
  "$push": {  
    "reviews": {  
      "_id": "58433009",  
      "listing_id": "10083468",  
      "reviewer_id": "reviewer_id_placeholder",  
      "reviewer_name": "Aqua",  
      "date": new Date(),  
      "comments": "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there."  
    }  
  }  
}
```

NOTE: I tried searching but you have to do it manually.

Filter: { "name": "Be Happy in Porto", "reviews.reviewer_name": "Aqua" }

Project: { "reviews._id": 1, "reviews.comments": 1 }

Statement 4 (checked)

USE to find owner (Should be 1)

Query: { "name": "Be Happy in Porto", "price": 30 }

Project: { name: 1, price: 1, "address.market": "1" }

USE to find in same area (Should be 554).

Query: { "address.market": "Porto" }

Project: { price: 1, "address.market": "1", name: 1 }

USE to find the number of properties in the Porto market with a price of 30 (16)

Query: { "address.market": "Porto", "price": 30 }

Sort: { price: 1 }

MongoDB Compass

Go to the aggregations tab. Make 5 stages.

Stage 1: make a \$match

```
{ name: "Be Happy in Porto" }
```

Stage 2: Make a \$lookup

```

{
  from: "listingsAndReviews",
  let: {
    property_type: "$property_type",
    bedrooms: "$bedrooms",
    market: "$address.market"
  },
  pipeline: [
    {
      $match: {
        $expr: {
          $and: [
            { $eq: [ "$property_type", "$$property_type" ] },
            { $eq: [ "$bedrooms", "$$bedrooms" ] },
            { $eq: [ "$address.market", "$$market" ] }
          ]
        }
      },
      { $group: { _id: null, avg_price: { $avg: "$price" } } }
    ],
    as: "avgPriceData"
  ]
}

```

Stage 3: Make a \$set stage.

```

{
  new_price: {
    $round: [
      { $multiply: [ { $arrayElemAt: [ "$avgPriceData.avg_price", 0 ] }, 0.9 ] }, 2
    ]
  }
}

```

Note: Creates or updates fields in the documents being processed in the pipeline. In this case, it's setting a new field called new_price.

- We calculate and store the result of the calculated discounted price.
- We round the result to 2 decimal places.
- We multiply 2 numbers, \$arrayElemAt and 0.9. This gives the 10% lower value.
- \$arrayElemAt. Looks at where we retrieve our values. We look at the array, avgPriceData calculated in stage 2, then the first value, index 0. There are no other values inside the index.

E.g.

*Let's say the average price of similar properties is **\$50**. Here's how the calculation proceeds in Stage 3:*

- **Average Price:** \$arrayElemAt: ["\$avgPriceData.avg_price", 0] returns **50**.
- **10% Discount:** \$multiply: [50, 0.9] results in **45**.
- **Rounding:** \$round: [45, 2] results in **45.00**.
- **Set New Price:** The new_price is set to **45.00**.

Stage 4: \$project

```
{
  _id: 1,
  price: "$new_price", // Only updating the price field
  name: 1,
  property_type: 1,
  bedrooms: 1,
  address: 1,
  // Add other necessary fields here
}
```

Note: Ensures that all the fields in the document except price are retained, so you only update price while keeping the rest intact.

Stage 5: Merge

```
{ into: "listingsAndReviews", on: "_id", whenMatched: "merge", whenNotMatched: "discard" }
```

Note: merging just the price field back into the document, preventing the deletion of other fields.

To search for updated price:

Query: { name: "Be Happy in Porto" }

Price: {price:1}

Below are wrong methods

Set	Stage 4: \$set <pre>//This is for \$Set { price: "\$new_price" }</pre> Note: I found that while this did run in the aggregation stage, it wasn't able to update the values in documents.
Update	Stage 4: Update <pre>{ _id: 1, // Keep the _id field for the update operation new_price: 1 // Keep the new price to apply it in the update }</pre> Then go filter: <pre>Filter: { name: "Be Happy in Porto" }</pre> Click update: <pre>{ \$set: { price: newPrice } //if the new price is 27, enter 27 }</pre> As a thought I thought we could update like we could previously (like the Netflix one) Don't do update. Otherwise you have to manually insert the listing id and new price value. - So you would have to remember the calculation in the Aggregation stage then manually update in the Document tab.

EXPLANATION

Stage 2: Here we look at other properties that are similar to the one in stage 1. We however add on to that by calculating the average price.

- Let are the matching conditions we want to pass onto pipeline.
- In the pipeline, operations are performed on the `listingsAndReviews` collection. We match the attributes from `$property_type` (outside of pipeline) to attributes inside pipeline, `$$property_type`.
- `Expressions` is to allow the usage of `$eq` to compare the fields.
- We then group by calculating the average price of filtered properties.
- The result is stored as `avgPriceData`.

Stage 3: A **new attribute** called `new_price` is created, which is calculated as 90% of the average price of similar properties. At this point, the price attribute is untouched.

Stage 4: During the merge operation, the original price is **updated** to the value of `new_price`, effectively setting the price to 10% lower than the average daily rate.

TO reset the price to do the Shell command:

Go to the aggregation tab. Make 4 stages.

Stage 1: match

```
{ name: "Be Happy in Porto" }
```

Stage 2: set

```
{ price: 30.00 }
```

Stage 3: project

```
{ _id: 1, price: 1 }
```

Stage 4: merge

```
{ into: "listingsAndReviews", on: "_id", whenMatched: "merge", whenNotMatched: "discard" }
```

To CHECK PRICE:

Query: { name: "Be Happy in Porto" }

Project: { price: 1 }

Shell

```
db.listingsAndReviews.aggregate([
  // Stage 1: Match the listing with the name "Be Happy in Porto"
  {
    $match: { name: "Be Happy in Porto" }
  },
  // Stage 2: Lookup to find similar properties with the same property_type, bedrooms, and market
  {
    $lookup: {
      from: "listingsAndReviews",
      let: { property_type: "$property_type", bedrooms: "$bedrooms", market: "$address.market" },
    },
    pipeline: [
      {
        $match: {
```

```

    $expr: {
      $and: [
        { $eq: [ "$property_type", "$$property_type" ] },
        { $eq: [ "$bedrooms", "$$bedrooms" ] },
        { $eq: [ "$address.market", "$$market" ] }
      ]
    }
  },
  {
    $group: { _id: null, avg_price: { $avg: "$price" } }
  },
  ],
  as: "avgPriceData"
}
},
// Stage 3: Set the new price to 10% lower than the average price
{
  $set: {
    new_price: {
      $round: [
        { $multiply: [ { $arrayElemAt: [ "$avgPriceData.avg_price", 0 ] }, 0.9 ] },
        2
      ]
    }
  }
},
// Stage 4: Project to update only the necessary fields
{
  $project: {
    _id: 1,
    price: "$new_price", // Only updating the price field
    name: 1, property_type: 1, bedrooms: 1, address: 1
  }
},
// Stage 5: Merge the result back into the original collection
{
  $merge: {
    into: "listingsAndReviews",
    on: "_id", whenMatched: "merge", whenNotMatched: "discard"
  }
}
])

```

Statement 5 (Checked)

Compass

First search for the review.

```
{
```

```
"reviews.comments": "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there."
```

```
}
```

Next update the comment with the specified comment.

```
{
```

```
  "$pull": {
```

```
    "reviews": {
```

```
      "comments": "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there."
```

```
    }
```

```
  }
```

```
}
```

NOTE: The \$pull operator removes the entire review object where the comment matches the given string.

Shell

```
db.listingsAndReviews.updateOne(
```

```
  { "reviews.comments": "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there." },
```

```
  {
```

```
    $pull: {
```

```
      reviews: {
```

```
        comments: "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there."
```

```
      }
```

```
    }
```

```
  }
```

```
);
```

TO CHECK, use this command:

```
{ "reviews.comments": "This holiday accommodation did not meet my expectation. Being in Portugal, I wanted to watch bull-fighting from the balcony. But, neither balcony nor bull-fighting nearby are there." }
```

Task 2: Extend the AirBnB database

Task 2.1 Embedded Approach

How I accommodate extending the database in a embedded approach is by first looking at the sample data. I can see that it provides a structure for a what booking will contain. I then query into the relevant documents, { name: "Be Happy in Porto" } and { name: "Be Happy in the Heart Of Montreal DT 1MIN to Metro" }. Only 2 documents are needed as bookings 1 and 2 share 1 document. No additional collections are needed, and I will only need to use listingsAndReviews for an embedded approach. This means that booking information is contained in each document. I just need to insert a new object, "bookings" which will contain all the sample data.

- With embedding, querying can be easier.

- We use embedding as many bookings(child documents) can correspond to one property (parent documents).

Task 2.2 Embedded Approach

Add the first booking in the sample bookings file to your extended database.

I just use the sample data but enclose it in shell by using updateOne and including the relevant filter for the booking.

Bookings 1

```
db.listingsAndReviews.updateOne(
  { name: "Be Happy in Porto" }, // Filter by listing name
  {
    $push: { // Add the new booking to the bookings array
      bookings: {
        booking_id: 12345,
        arrival_date: ISODate("2025-03-11T14:00:00.000+10:00"),
        departure_date: ISODate("2025-03-13T10:00:00.000+10:00"),
        client: {
          name: "Brodie Mackrell",
          email: "BrodieMackrell@rhyta.com",
          daytime_phone: "(03) 5352 6218",
          mobile: "0453 526 218",
          postal_address: "48 Mills Street DALYSTON VIC 3992",
          home_address: "48 Mills Street DALYSTON VIC 3992"
        },
        deposit_paid: 10,
        balance_due: 20,
        balance_due_date: ISODate("2025-03-01T00:00:00.000+10:00"),
        number_of_guests: 2,
        guests: [
          { name: "Brodie", age: 44 },
          { name: "John", age: 44 }
        ]
      }
    }
  }
);
```

Booking 2

```
db.listingsAndReviews.updateOne(
  { name: "Be Happy in Porto" }, // Same listing name for second booking
  {
    $push: { // Add the second booking to the bookings array
      bookings: {
        booking_id: 12347,
        arrival_date: ISODate("2025-11-11T14:00:00.000+11:00"),
        departure_date: ISODate("2025-11-14T10:00:00.000+11:00"),
        client: {
```

```

    name: "Alexander Kevin",
    email: "AlexanderKevin@teleworm.us",
    daytime_phone: "(08) 9038 8742",
    mobile: "0490 388 742",
    postal_address: "81 Farrar Parade CARNAMAH WA 6517",
    home_address: "81 Farrar Parade CARNAMAH WA 6517"
  },
  deposit_paid: 30,
  balance_due: 0,
  balance_due_date: null,
  number_of_guests: 3,
  guests: [
    { name: "Alexander", age: 54 },
    { name: "Alexa", age: 50 },
    { name: "Siri", age: 15 }
  ]
}
}
);
Booking 3
db.listingsAndReviews.updateOne(
  { name: "Be Happy in the Heart Of Montreal DT 1MIN to Metro" }, // Filter by different listing
  name
  {
    $push: { // Add the booking to the bookings array
      bookings: {
        booking_id: 12346,
        arrival_date: ISODate("2025-04-11T14:00:00.000+10:00"),
        departure_date: ISODate("2025-04-12T10:00:00.000+10:00"),
        client: {
          name: "Brodie Mackrell",
          email: "BrodieMackrell@rhyta.com",
          daytime_phone: "(03) 5352 6218",
          mobile: "0453 526 218",
          postal_address: "48 Mills Street DALYSTON VIC 3992",
          home_address: "48 Mills Street DALYSTON VIC 3992"
        },
        deposit_paid: 20,
        balance_due: 59,
        balance_due_date: ISODate("2025-04-01T00:00:00.000+10:00"),
        number_of_guests: 3,
        guests: [
          { name: "Brodie", age: 44 },
          { name: "John", age: 44 },
          { name: "John Jr", age: 10 }
        ]
      }
    }
  }
);

```

Display the client name, email address, booking start date, and booking end date for all bookings made so far for the listing named “Be Happy in Porto”.

Query: {name: "Be Happy in Porto"}

Project:

```
{"bookings.client.name":1,"bookings.client.email":1,"bookings.arrival_date":1,"bookings.departure_date":1}
```

Remove bookings for referenced approach

//Booking 1

```
db.listingsAndReviews.updateOne(
  { name: "Be Happy in Porto" }, // Filter by listing name
  {
    $pull: { // Remove the booking with the specified booking_id
      bookings: { booking_id: 12345 }
    }
  }
);
```

//Booking 2

```
db.listingsAndReviews.updateOne(
  { name: "Be Happy in Porto" }, // Same listing name for the second booking
  {
    $pull: { // Remove the booking with the specified booking_id
      bookings: { booking_id: 12347 }
    }
  }
);
```

REFRESH then delete the bookings field

```
db.listingsAndReviews.updateOne(
  { name: "Be Happy in Porto", bookings: { $size: 0 } }, // Filter by listing name and check if bookings is empty
  {
    $unset: { bookings: "" } // Remove the bookings field if it's empty
  }
);
```

//Booking 3

```
db.listingsAndReviews.updateOne(
  { name: "Be Happy in the Heart Of Montreal DT 1MIN to Metro" }, // Filter by the listing name
  {
    $pull: { // Remove the booking with the specified booking_id
      bookings: { booking_id: 12346 }
    }
  }
);
```

Refresh then unset

```
db.listingsAndReviews.updateOne(
```

```
{ name: "Be Happy in the Heart Of Montreal DT 1MIN to Metro", bookings: { $size: 0 } }, // Filter
by listing name and check if bookings is empty
{
  $unset: { bookings: "" } // Remove the bookings field if it's empty
}
);
```

Task 2.1: Referenced Approach

I would create the collection, Client. This is to store information documents about the clients. This is because a client can be involved in many bookings. In this case it is Brodie, as he is involved in booking 1 and 3. We create this document to avoid redundancy by normalising the fields like relational model.

- In our case we are only focusing on 3 documents which will not affect performance.

I would create new collection in for Bookings. This is to store the 3 bookings found in the sample data. The Booking collection links to the collection listAndReviews by using _id of it (called listing_id in the Booking Collection). Bookings links to Client collection by using the object id that client uses.

This is to avoid redundancy by normalising the fields similar to relational model.

- While a referenced approach can lead to redundancy, and performance loss, we are only focusing on 3 documents so issues would be minimal.

Task 2.2: Referenced approach

Add the first booking in the sample bookings file to your extended database.

Only after creating the Client documents and getting their ID's do you create the Booking collection. To do that go into the database, "Create Collection" then enter "Clients".

- We create Client first as it is less independent.
- Then "Add Data" then "Insert Document".

```
//Client documents
//Client 1
{
  "_id": {
    "$oid": "670232f5563825de0da55f81"
  },
  "name": "Brodie Mackrell",
  "email": "BrodieMackrell@rhyta.com",
  "daytime_phone": "(03) 5352 6218",
  "mobile": "0453 526 218",
  "postal_address": "48 Mills Street DALYSTON VIC 3992",
  "home_address": "48 Mills Street DALYSTON VIC 3992"
}
//Client 2
{
  "_id": {
    "$oid": "67023660563825de0da55f83"
  },
  ,
```



```

"name": "Alexander Kevin",
"email": "AlexanderKevin@teleworm.us",
"daytime_phone": "(08) 9038 8742",
"mobile": "0490 388 742",
"postal_address": "81 Farrar Parade CARNAMAH WA 6517",
"home_address": "81 Farrar Parade CARNAMAH WA 6517"
}

```

Next create the Booking documents in the Bookings collection.

```
/**
```

This is booking 1

```
**/
```

```

{
  "_id": {
    "$oid": "670238f3563825de0da55f89"
  },
  "booking_id": "12345",
  "arrival_date": {
    "$date": "2025-03-11T04:00:00.000Z"
  },
  "departure_date": {
    "$date": "2025-03-13T00:00:00.000Z"
  },
  "listing_id": "10083468",
  "client_id": {
    "$oid": "670232f5563825de0da55f81"
  },
  "deposit_paid": 10,
  "balance_due": 20,
  "balance_due_date": {
    "$date": "2025-02-28T14:00:00.000Z"
  },
  "number_of_guests": 2,
  "guests": [
    {
      "name": "Brodie",
      "age": "44"
    },
    {
      "name": "John",
      "age": "44"
    }
  ]
}

```

// Insert Booking 2

```

{
  "_id": {
    "$oid": "67023d00563825de0da55f8c"
  }
}

```

```

},
"booking_id": 12347,
"arrival_date": {
  "$date": "2025-11-11T03:00:00.000Z"
},
"departure_date": {
  "$date": "2025-11-13T23:00:00.000Z"
},
"listing_id": "10083468",
"client_id": {
  "$oid": "67023660563825de0da55f83"
},
"deposit_paid": 30,
"balance_due": 0,
"balance_due_date": "",
"number_of_guests": 3,
"guests": [
  {
    "name": "Alexander",
    "age": "54"
  },
  {
    "name": "Alexa",
    "age": "50"
  },
  {
    "name": "Siri",
    "age": "15"
  }
]
}

```

```

/**
This is booking document 3
**/
{
  "_id": {
    "$oid": "67023f6f563825de0da55f90"
  },
  "booking_id": 12346,
  "arrival_date": {
    "$date": "2025-04-11T04:00:00.000Z"
  },
  "departure_date": {
    "$date": "2025-04-12T00:00:00.000Z"
  },
  "listing_id": "25937713",
  "client_id": {
    "$oid": "670232f5563825de0da55f81"
  }
}

```

```

},
"deposit_paid": 20,
"balance_due": 59,
"balance_due_date": {
  "$date": "2025-03-31T14:00:00.000Z"
},
"number_of_guests": 3,
"guests": [
  {
    "name": "Brodie",
    "age": "44"
  },
  {
    "name": "John",
    "age": "44"
  },
  {
    "name": "John Jr",
    "age": "10"
  }
]
}

```

Display the client name, email address, booking start date, and booking end date for all bookings made so far for the listing named “Be Happy in Porto”.

Within the aggregation tab make the following stages.

Stage 1: match

```
{ name: "Be Happy in Porto"}
```

Stage 2: lookup

```
{ from: "Bookings", localField: "_id", foreignField: "listing_id", as: "bookings"}
```

Stage 3: unwind

```
"$bookings"
```

Stage 4:lookup

```
{ from: "Clients", localField: "bookings.client_id", foreignField: "_id", as: "client_info"}
```

Stage 5: unwind

```
"$client_info"
```

Stage 6: project

```

{
  "client_info.name": 1, "client_info.email": 1,
  "bookings.arrival_date": 1, "bookings.departure_date": 1
}

```

References

GeeksforGeeks (2024) *MongoDB Cheat Sheet*, GeeksforGeeks, accessed 6 October 2024.
<https://www.geeksforgeeks.org/mongodb-cheat-sheet/>

MongoDB (2024) *db.collection.insert()*, accessed 6 October 2024.
<https://www.mongodb.com/docs/manual/reference/method/db.collection.insert/>

w3schools (2024) *MongoDB mongosh Update*, accessed 6 October 2024.
https://www.w3schools.com/mongodb/mongodb_mongosh_update.php

MongoDB (2024) *\$addToSet*, MongoDB, accessed 6 October 2024.
<https://www.mongodb.com/docs/manual/reference/operator/update/addToSet/>

1B

3T (2024) *The Beginner's Guide to MongoDB Aggregation (With Exercise)*, 3T, accessed 6 October 2024. <https://studio3t.com/knowledge-base/articles/mongodb-aggregation-framework/>

MongoDB (2024) *\$all*, MongoDB, accessed 7 October 2024.
<https://www.mongodb.com/docs/manual/reference/operator/query/all/>

1C

MongoDB (2024) *\$insert (aggregation)*, MongoDB, accessed 6 October 2024.
<https://www.mongodb.com/docs/manual/reference/method/db.collection.insert/>

1D

MongoDB (2024) *\$avg (aggregation)*, MongoDB, accessed 6 October 2024.
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/avg/>

1E

MongoDB (2024) *\$unwind (aggregation)*, MongoDB, accessed 6 October 2024.
https://www.mongodb.com/docs/manual/reference/operator/aggregation/unwind/?utm_source=compass&utm_medium=product

MongoDB (2024) *\$project (aggregation)*, MongoDB, accessed 6 October 2024.
https://www.mongodb.com/docs/manual/reference/operator/aggregation/project/?utm_source=compass&utm_medium=product

1F

MongoDB (2024) *\$sort (aggregation)*, MongoDB, accessed 6 October 2024.
<https://www.mongodb.com/docs/manual/reference/operator/aggregation/sort/>